Università
degli Studi
di Padova

Wavelet-based image compression

Marco Cagnazzo

Multimedia Communications

Outline

Outline

Signal Analysis through Projections

Universal Principle

The linear transforms used in signal processing and multimedia compression (DFT, DCT, Wavelet) are defined by the projection of the input signal onto an appropriate set of basis functions.

Given an orthonormal basis $\{\phi_k(t)\}$, any signal $x(t)$ can be perfectly represented as a linear combination:

$$x(t) = \sum_k c_k \cdot \phi_k(t) \tag{1}$$

The Transform as a Scalar Product

The coefficients c_k are the result of the **scalar product**, which quantifies the similarity between the signal and each specific basis function:

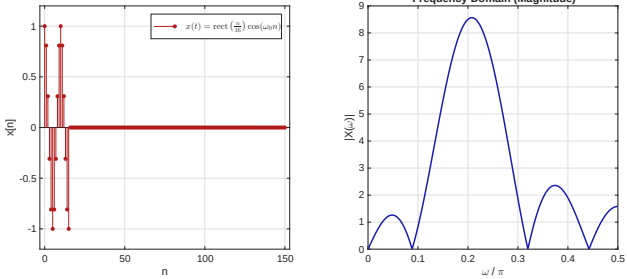
Transform Coefficients

$$c_k = \langle x(t), \phi_k(t) \rangle = \int_{-\infty}^{+\infty} x(t) \cdot \phi_k^*(t)\, dt$$

- Large $|c_k|$: High similarity (the signal shares significant energy with that component).
- The transform is essentially a **change of basis** to achieve energy compaction (sparsity).

Resolution Trade-off: Windowed Pure Tone

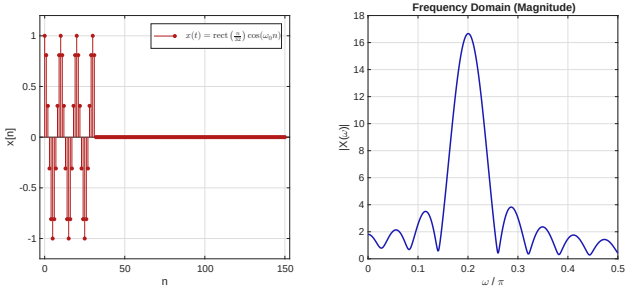
Consider a signal $x[n] = \cos(\omega_0 n) \cdot \text{rect}_N[n]$. What happens to our knowledge of frequency and time as we change the window length N?



Short Window ($N = 16$): Excellent time localization, but frequency peak is very blurry ($\Delta\omega$ is large).

Resolution Trade-off: Windowed Pure Tone

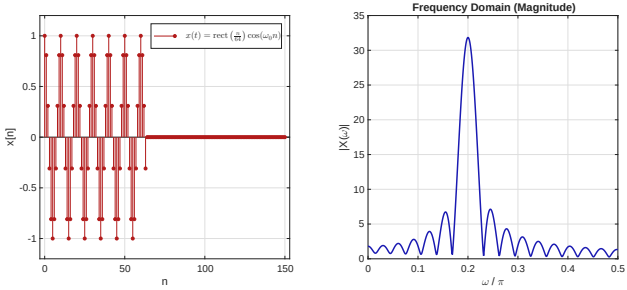
Consider a signal $x[n] = \cos(\omega_0 n) \cdot \text{rect}_N[n]$. What happens to our knowledge of frequency and time as we change the window length N?



Medium Window ($N = 32$): Frequency resolution begins to improve, but time uncertainty increases.

Resolution Trade-off: Windowed Pure Tone

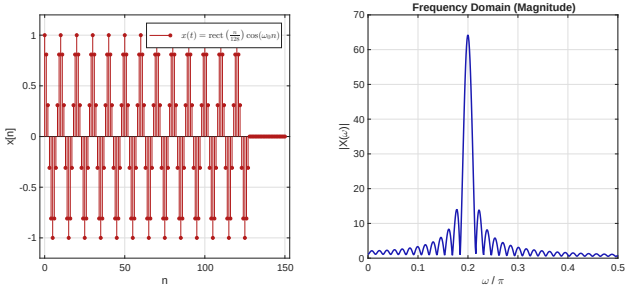
Consider a signal $x[n] = \cos(\omega_0 n) \cdot \text{rect}_N[n]$. What happens to our knowledge of frequency and time as we change the window length N?



Long Window ($N = 64$): Frequency peak becomes sharper, but the signal is now spread over more time.

Resolution Trade-off: Windowed Pure Tone

Consider a signal $x[n] = \cos(\omega_0 n) \cdot \text{rect}_N[n]$. What happens to our knowledge of frequency and time as we change the window length N?



Very Long Window ($N = 128$): Excellent frequency resolution ($\Delta\omega \approx 0$), but poor time localization (Δt is large).

The Time-Frequency Uncertainty Principle

The Fundamental Limit

There is an **inverse relationship** between time resolution (Δt) and frequency resolution (Δf). We cannot simultaneously increase both.

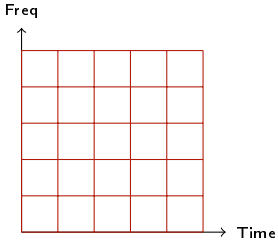
According to the **Heisenberg-like Uncertainty Principle**:

$$\Delta t \cdot \Delta f \geq \frac{1}{4\pi} \tag{2}$$

- The area of the "uncertainty cell" in the time-frequency plane is **constant**.
- We can only change the **shape** of the cell (narrow/tall vs. wide/flat).

Frequency Analysis: STFT and Rigid Tiling

Standard transforms as JPEG's Block-DCT use a **fixed window size** for all frequencies (8×8 blocks).



The Problem

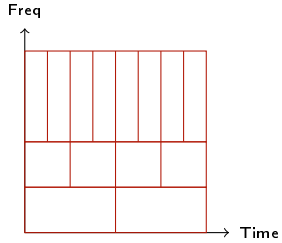
Once N is chosen, the resolution is constant. We are forced to choose between seeing **fast transients** (small N) or **precise frequencies** (large N).

Natural signals (images, audio) contain **both** and thus require a flexible approach.

Wavelets: Adaptive Multiresolution

Wavelets solve the trade-off by adapting the window shape to the frequency:

Wavelet Tiling



- **High Freq:** Short windows (fine time res) to catch edges/anomalies.
- **Low Freq:** Long windows (fine freq res) to catch trends/textures.

Biological Link: This matches how the human visual and auditory systems process information.

Mechanism: Shift and Dilation

To achieve adaptive tiling, we use a single **mother wavelet** $\psi(t)$ and generate the basis through scaling and translation:

The Wavelet Basis

$$\psi_{a,b}(t) = \frac{1}{\sqrt{a}}\psi\left(\frac{t-b}{a}\right)$$

Recall the **Scaling Property** of the Fourier Transform:

Time Scaling vs. Frequency Scaling

If $\mathcal{F}\{x(t)\} = X(f)$, then $\mathcal{F}\left\{x\left(\frac{t}{a}\right)\right\} = |a|X(af)$

Mechanism: Shift and Dilation

- **Large a (Dilation):** The wavelet stretches in time and **shrinks in frequency** (lower frequencies, finer Δf).
- **Small a (Compression):** The wavelet shrinks in time and **stretches in frequency** (higher frequencies, finer Δt).

Wavelets and images : Motivations

Image model:
trends +
anomalies

Wavelets and images : Motivations

Image model:
trends +
anomalies

Wavelets and images : Motivations

Image model:
trends +
anomalies

Wavelets and images : Motivations

- *Anomalies* :
  - Abrupt variations of the signal
  - High frequency contributions
  - Objects' contours
  - Good spatial resolution
  - Rough frequency resolution
- *Trends* :
  - Slow variations of the signal
  - Low frequency contributions
  - Objects' texture
  - Rough spatial resolution
  - Good frequency resolution

Wavelets and images : Motivations

Signal model: an image row

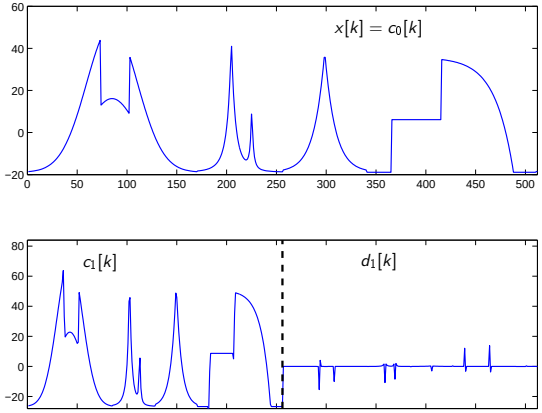
Wavelets and images : Motivations

Signal model: an image row

Wavelets and images : Motivations

Signal model: an image row

Wavelets and Multiple resolution analysis

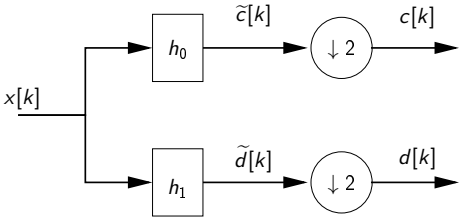


- Approximation: low resolution version
- "Details": zeros when the signal is polynomial

Outline

1D filter banks

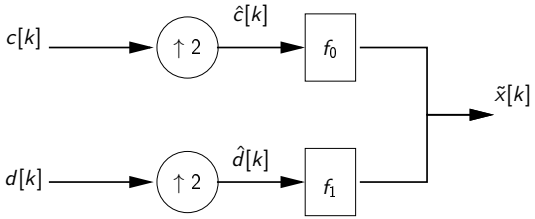
Decomposition



Analysis filter bank

$2\downarrow$: decimation : $c[k] = \widetilde{c}[2k]$

Reconstruction



Synthesis filter bank

$2\uparrow$: interpolation operator, doubles the sample number

$$\hat{c}[k] = \begin{cases} c[k/2] & \text{if } k \text{ is even} \\ 0 & \text{if } k \text{ is odd} \end{cases}$$

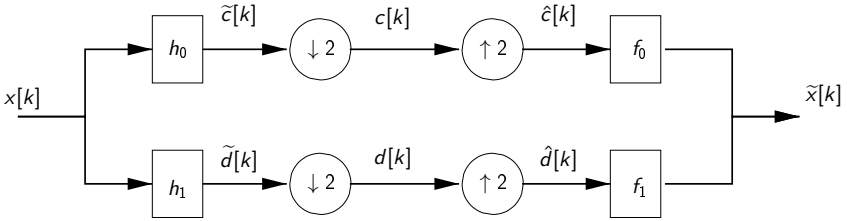
Filter properties

- Perfect reconstruction (PR)
- FIR
- Orthogonality
- Vanishing moments
- Symmetry

Perfect reconstruction conditions

We want PR after synthesis and analysis filter banks :
$\forall k \in \mathbb{Z}$,

$$\widetilde{x}_k = x_{k+\ell} \Longleftrightarrow \widetilde{X}(z) = z^{-\ell} X(z)$$



Z-domain relationships

filter
$$\widetilde{C}(z) = \sum_{n=-\infty}^{\infty} \widetilde{c}_n z^{-n} = H_0(z) X(z)$$

decimation
$$C(z) = \frac{1}{2}\left[\widetilde{C}\left(z^{1/2}\right) + \widetilde{C}\left(-z^{1/2}\right)\right]$$

interpolation
$$\hat{C}(z) = C\left(z^2\right)$$

output
$$\widetilde{X}(z) = F_0(z) C\left(z^2\right) + F_1(z) D\left(z^2\right)$$

$$\widetilde{X}(z) = \frac{1}{2}\left[F_0(z) H_0(z) + F_1(z) H_1(z)\right] X(z) + \frac{1}{2}\left[F_0(z) H_0(-z) + F_1(z) H_1(-z)\right] X(-z)$$

PR conditions in Z

$\forall k \in \mathbb{Z}$,

$$\widetilde{x}_k = x_{k+\ell} \Longleftrightarrow \widetilde{X}(z) = z^{-\ell} X(z)$$

$$\Updownarrow$$

$$T(z) = H_0(z) F_0(z) + H_1(z) F_1(z) = 2z^{-\ell} \quad \text{Non distortion (ND)}$$

$$A(z) = H_0(-z) F_0(z) + H_1(-z) F_1(z) = 0 \quad \text{Aliasing cancelation (AC)}$$

Perfect reconstruction conditions

Matrix form

If the analysis filter bank is given, the synthesis one is determined by:

$$\begin{bmatrix} H_0(z) & H_1(z) \\ H_0(-z) & H_1(-z) \end{bmatrix} \cdot \begin{bmatrix} F_0(z) \\ F_1(z) \end{bmatrix} = \begin{bmatrix} 2z^{-\ell} \\ 0 \end{bmatrix}$$

We need the *modulation matrix* to be invertible, ie.

$$\forall z \in \mathbb{C} : |z| = 1, \quad \Delta(z) = H_0(z)H_1(-z) - H_1(z)H_0(-z) \neq 0$$

Orthogonality

QMF and CQF are orthogonal, which assures energy conservation:

$$\sum_{k=-\infty}^{\infty} (x_k)^2 = \sum_{k=-\infty}^{\infty} (c_k)^2 + \sum_{k=-\infty}^{\infty} (d_k)^2$$

$\Rightarrow$ reconstruction error = quantization error on DWT coefficients
For bi-orthogonal filters, the reconstruction errors is a weighted sum of the quantization errors on the DWT subbands, with suitable weights ω_i

Vanishing moments

- Vanishing moments (VM) represent filter ability to reproduce polynomials: a filter with p VM can represent polynomials with degree $< p$
- The High-pass filter will not respond to a polynomial input with degree $< p$
- In this case all the signal information is preserved in the approximation signal (half the samples)
- A filter with p VM has at least $2p$ taps

Borders problem

- The filter bank properties we have seen are valid for infinite-size signals
- We are interested in finite support signals
- How to interpret the previous results for finite support signals?

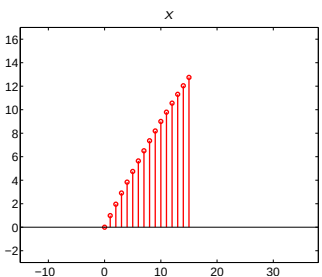
Borders problem

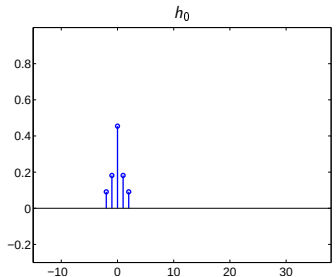
- The filter bank properties we have seen are valid for infinite-size signals
- We are interested in finite support signals
- How to interpret the previous results for finite support signals?
- Zero padding would introduce a coefficient expansion
- Filtering an N-size signal with an M-size produces a signal with size $N + M - 1$

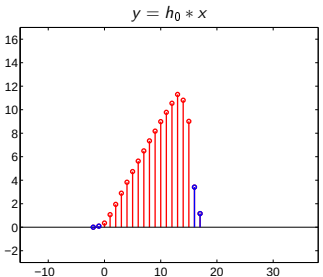
Borders problem

- The filter bank properties we have seen are valid for infinite-size signals
- We are interested in finite support signals
- How to interpret the previous results for finite support signals?
- Zero padding would introduce a coefficient expansion
- Filtering an N-size signal with an M-size produces a signal with size $N + M - 1$
- Periodization?
- Symmetrization?

Borders problem: Coefficient expansion



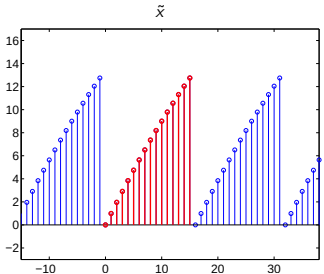


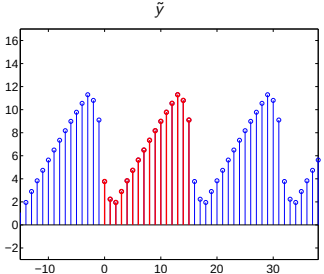


Borders problem: Periodization

- A signal x of support N is considered as a periodic signal $\tilde{x}$ of period N
- Filtering $\tilde{x}$ with h_0 results into a periodic output $\tilde{y}$
- $\tilde{y}$ has the same period N as $\tilde{x}$
- So we need to compute just N samples of $\tilde{y}$
- However, periodization introduces "jumps" in a regular signal

Borders problem: Periodization





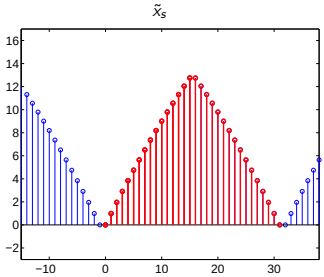
Borders problem: Symmetry

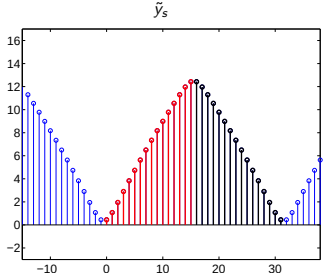
- Symmetrization before periodization reduces the impact on signal regularity
- But it doubles the number of coefficients...

Borders problem: Symmetry

- Symmetrization before periodization reduces the impact on signal regularity
- But it doubles the number of coefficients...
- Unless the filters are symmetric, too
  - We use x as half-period of $\tilde{x}_s$
  - $\tilde{x}_s$ has a period of $2N$ samples
  - Filtering $\tilde{x}_s$ with h_0, produces $\tilde{y}_s$
  - If h_0 is symmetric, $\tilde{y}_s$ is periodic and symmetric, with period $2N$: we only need to compute N samples

Borders problem: Symmetry





Haar filter

$$h_0(k) = \boxed{1}\,\boxed{1} \qquad f_0(k) = \boxed{1}\,\boxed{1}$$

$$h_1(k) = \boxed{1}\,\boxed{-1} \qquad f_1(k) = \boxed{-1}\,\boxed{1}$$

- Symmetric
- Orthogonal (Haar is both QMF and CQF)
- VM = 1
  - Only capable to represent piecewise constant signals

Summary: perfect reconstruction and borders

- Linear convolution leads to coefficient expansion
- Solution: circular convolution
  - Circular convolution allows to reconstruct an N-samples signal with N wavelet coefficients
  - The periodization generates borders discontinuities, i.e. spurious high frequencies coefficients that demand a lot of coding resources
- Solution: Symmetric periodization
  - No discontinuities
  - Does it double the coefficient number?
  - No, if the filter is **symmetric**!

Bad news: the only orthogonal symmetric FIR filter is Haar!

Biorthogonal filters

Cohen-Daubechies-Fauveau filters

With biorthogonal filters, if h_0 has p VM and f_0 has $\widetilde{p}$ VM, the filter has at least $p + \widetilde{p} - 1$ taps.

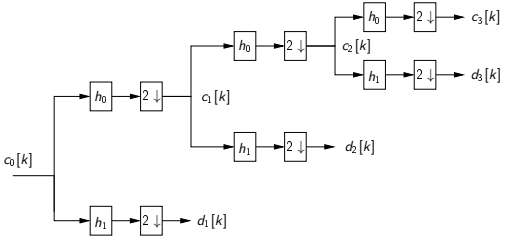
The CDF filters have the following properties:

- They are symmetric (linear phase)
- They maximize the VM for a given filter length
- They are close to orthogonality (weights ω_i are close to one)

They are the most widely adopted filters in image compression techniques

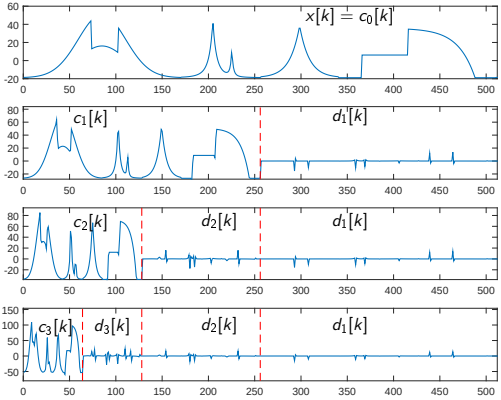
1D Multiresolution analysis

Decomposition

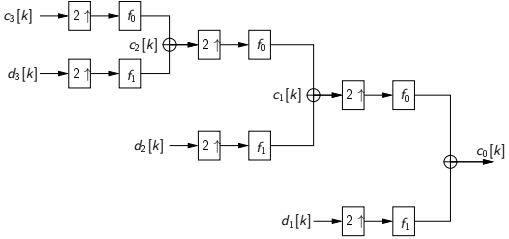


Three levels wavelet decomposition structure

Multiresolution Analysis 1D



Reconstruction



Reconstruction from wavelet coefficients

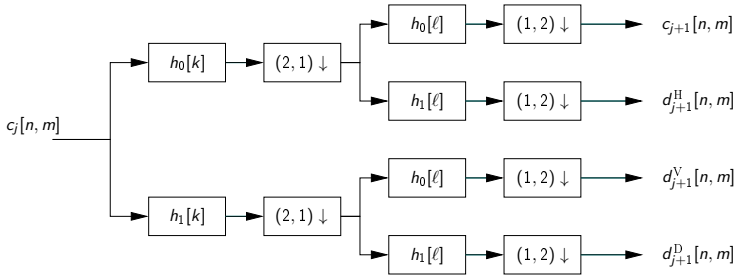
Recap of the last lesson

- "Trends+anomalies" model for visual signals
- A filterbank (LPF, HPF, subsampling) can separate approximation and details, inducing signal sparsification
- Recursive analysis of the approximation can further sparsify the signal
- It is possible to find FIR filterbanks with perfect reconstruction and any number of vanishing moments (i.e., ability to remove polynomial parts of the signal)
- However we must choose between symmetry (which avoids frequency leaking) and orthogonality: for compression symmetry is more important
- The most compact filters achieving these properties are the Daubechies filters
- In image compression, 2 filterbanks are used:
  - Daubechies 9/7: four vanishing moments, **best** fit for image compression
  - Daubechies 5/3: two vanishing moments, but **integer-valued** filter taps, allowing for error-free reconstruction
- The filterbanks are equivalent to a linear transform (matrix multiplication), referred to as **Discrete Wavelet Transform** (DWT)

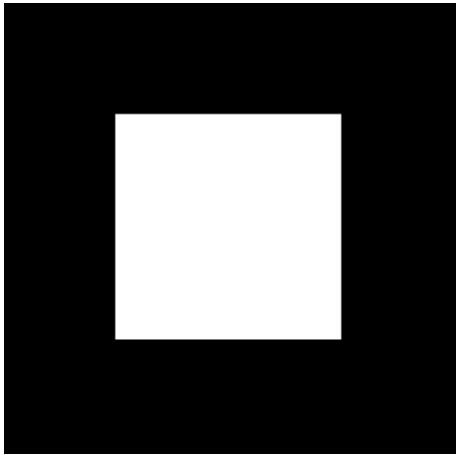
2D AMR

2D Filter banks for separable transform

One decomposition level

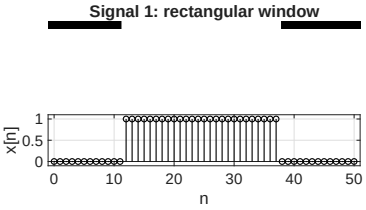


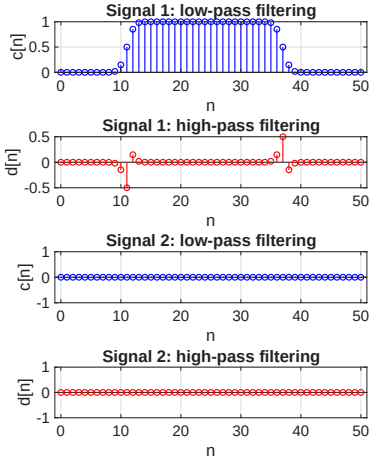
Example: synthetic image



We consider a synthetic image: a white square over a black background. Each row and column of this image is either a null signal or a rectangular window. What happens when we perform LP/HP filtering along the rows?

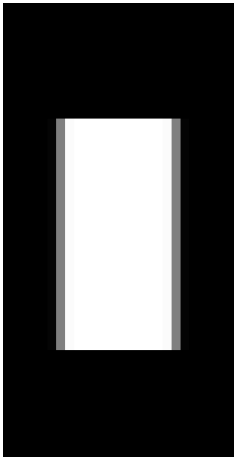
Example: synthetic image

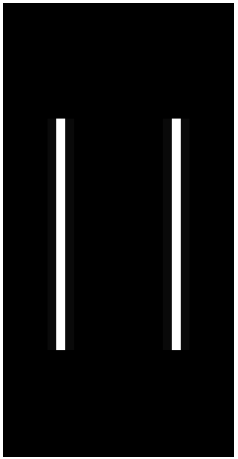




Let us first consider the central rows: the signal is a rectangular window; low-pass filtering smooths the edges, keeping the general behavior; high-pass filtering only responds on edges.
When the input is zero, both LP and HP respond with zero.
Let us subsample and show the result.

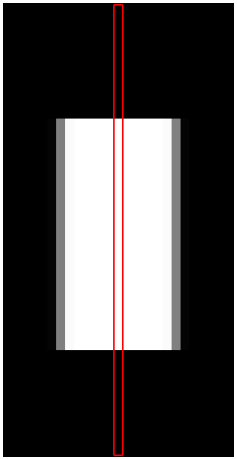
Example: synthetic image

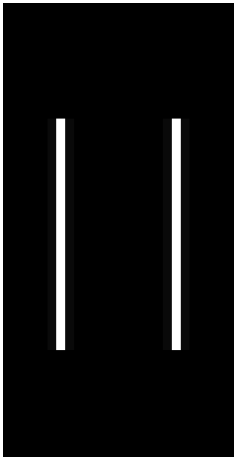




The LP subband represents the signal at halved horizontal resolution, while the HP subband shows the vertical contours (high-frequency contributions along the horizontal directio)n. We observe that the column of the new image are again:

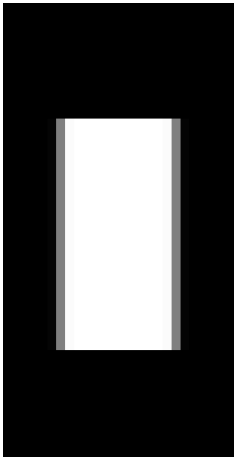
Example: synthetic image

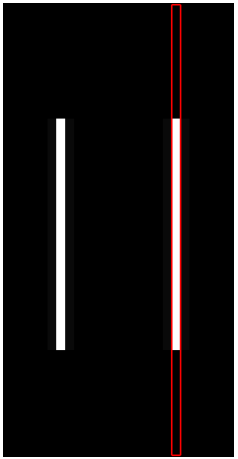




The LP subband represents the signal at halved horizontal resolution, while the HP subband shows the vertical contours (high-frequency contributions along the horizontal directio)n. We observe that the column of the new image are again: either **rectangular windows**

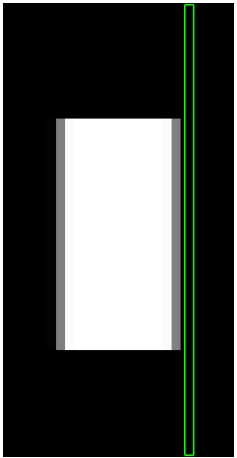
Example: synthetic image

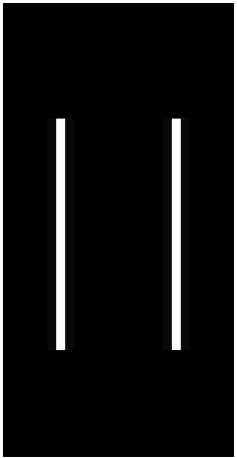




The LP subband represents the signal at halved horizontal resolution, while the HP subband shows the vertical contours (high-frequency contributions along the horizontal directio)n. We observe that the column of the new image are again: either **rectangular windows**

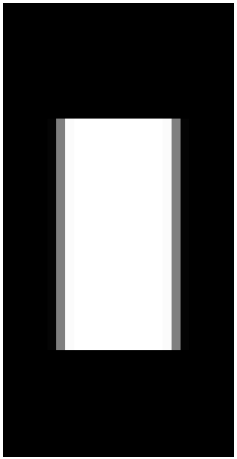
Example: synthetic image

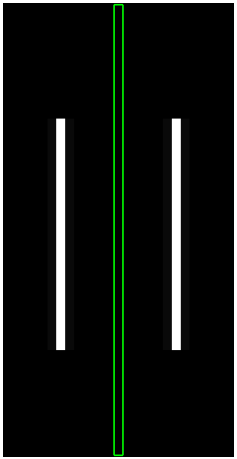




The LP subband represents the signal at halved horizontal resolution, while the HP subband shows the vertical contours (high-frequency contributions along the horizontal directio)n. We observe that the column of the new image are again: either **rectangular windows** or **null signals**

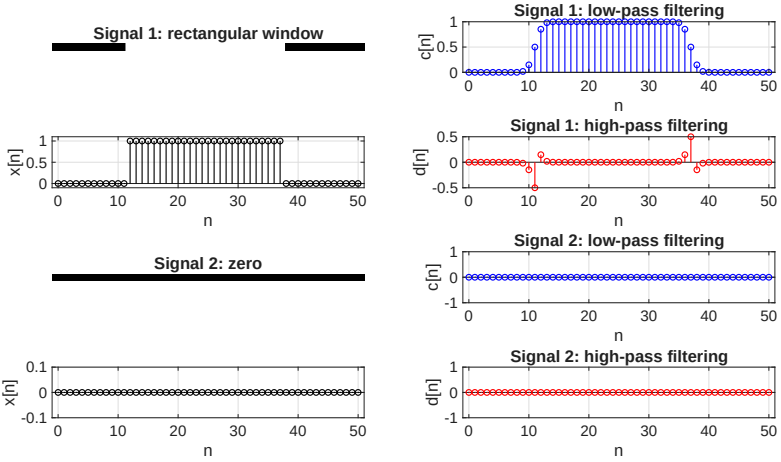
Example: synthetic image





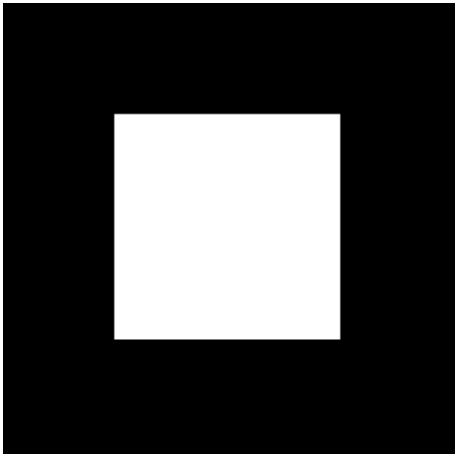
The LP subband represents the signal at halved horizontal resolution, while the HP subband shows the vertical contours (high-frequency contributions along the horizontal directio)n. We observe that the column of the new image are again: either **rectangular windows** or **null signals**

Example: synthetic image



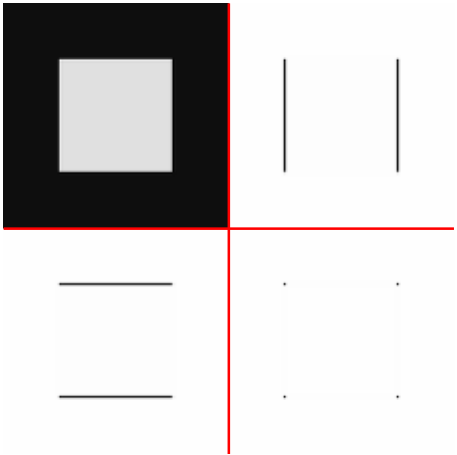
When we filter the columns we obtain again either smoothed rectangles or impulses at the edges

Example: synthetic image



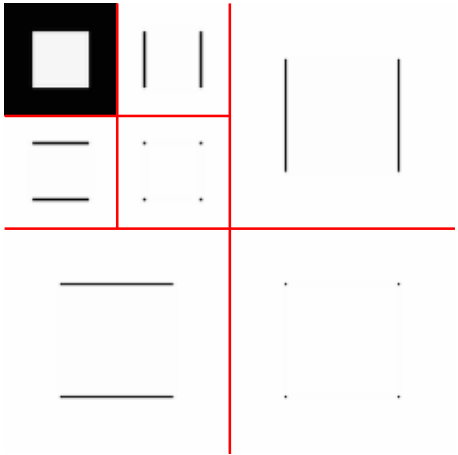
Let us show the first level of MRA: note that, to improve image visibility, color scale in the **detail subbands** is reversed:
white = zero, black = large.

Example: synthetic image



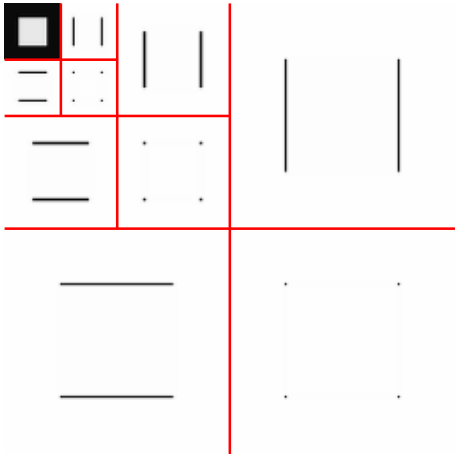
In the top-left corner (**LL subband**) we find the low-resolution version of the original image, obtained by low-pass filtering both rows and columns; in the top-right corner (**HL subband**) we find the horizontal details (high-pass on rows, low-pass on cols; in the bottom-left (**LH subband**) we find vertical details (low-pass on rows, high-pass on cols; and finally in the bottom-right (**HH subband**) the diagonal details.

Example: synthetic image



Since the LL subbands is a low-resolution version of the original image, we can apply another level of MRA to it: we obtain a two-levels discrete wavelet transform. Here the signal is even sparser (only the LL subband and some detail coefficients are non-zero).

Example: synthetic image

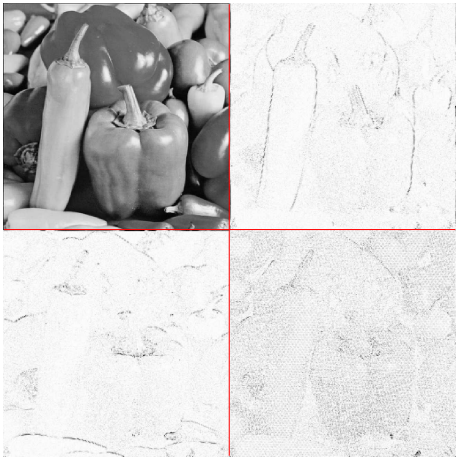


Three MRA levels: the non zero-coefficients are 1/64 of the original, plus "a few" detail coefficients.

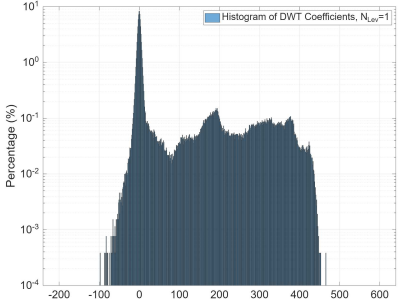
Example: natural image

As usual, the natural image is not sparse.

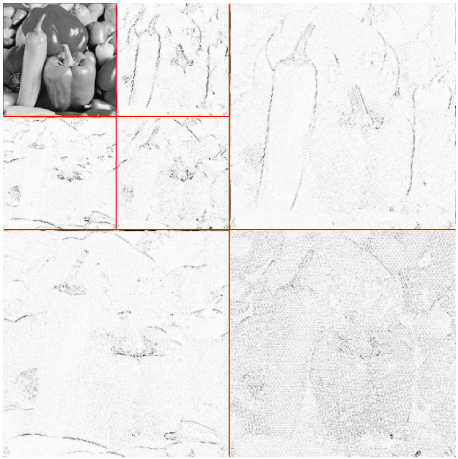
Example: natural image



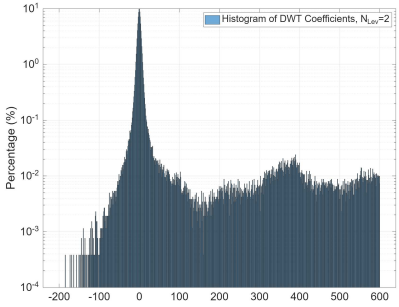
After one level of 2D-DWT, we achieve some sparsity, but still 1/4 of coeff's (LL subband) are relatively large.



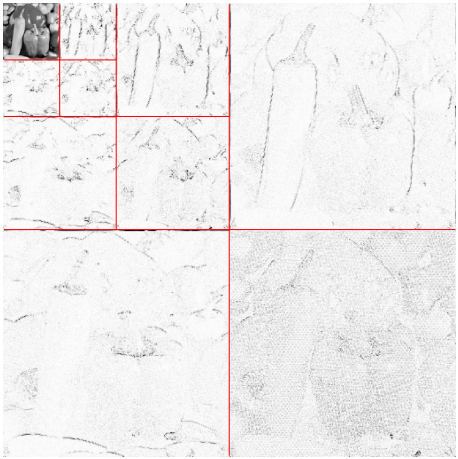
Example: natural image



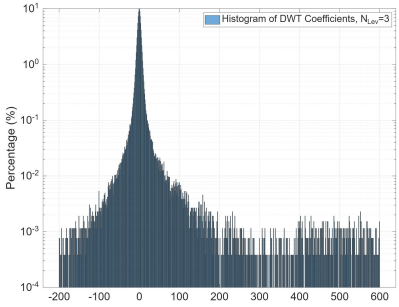
With 2 levels, the number of large coefficients drops dramatically.



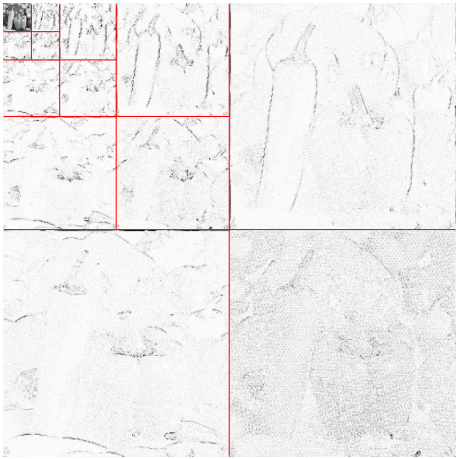
Example: natural image



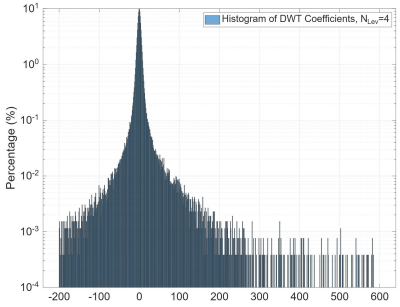
This trend continues with 3 to 5 levels of decomposition.



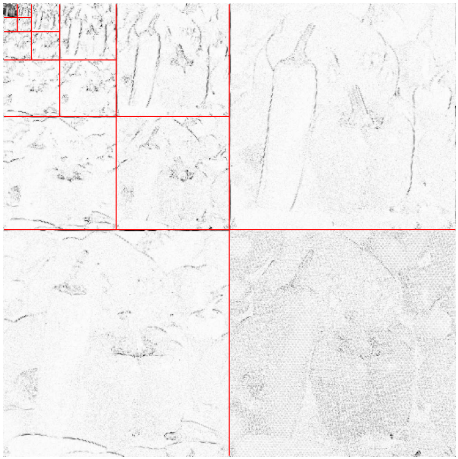
Example: natural image



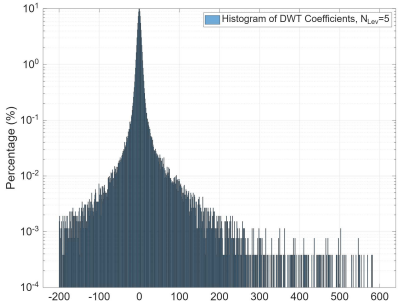
This trend continues with 3 to 5 levels of decomposition.



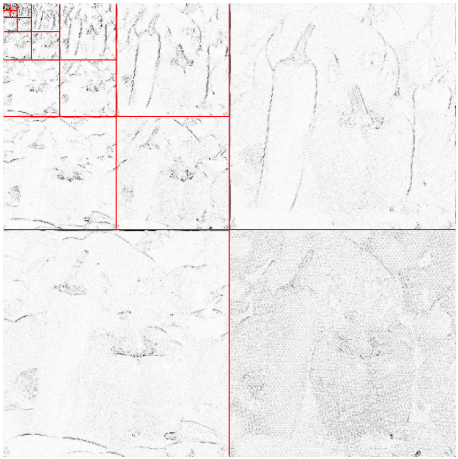
Example: natural image



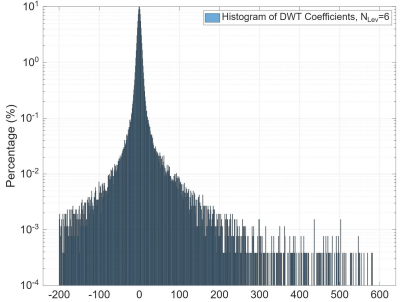
This trend continues with 3 to 5 levels of decomposition.



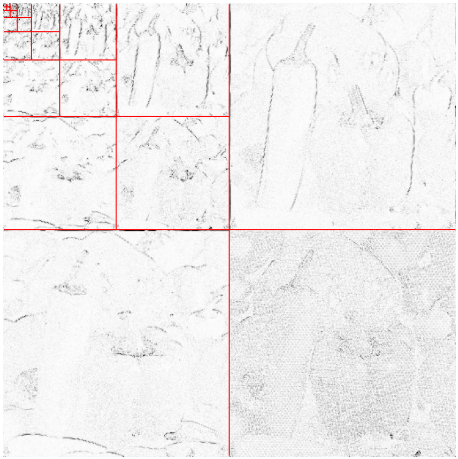
Example: natural image



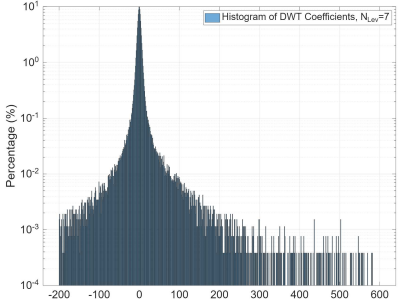
Further increasing the number of decomposition levels usually does not bring significant improvement in sparsity



Example: natural image



Further increasing the number of decomposition levels usually does not bring significant improvement in sparsity



Choosing the number of decomposition levels

Typically, 4 to 6 levels of 2D-DWT is the best choice. Indeed, further increasing the number of levels is often ineffective:

- The LL subband is smaller and smaller, thus a new level impacts only a few coefficients
- The low-resolution subband tends to loose the spatial smoothness: transforming an irregular signal does not improve sparsity since the trend+anomalies model does not hold any longer

Outline

Compression with DWT

- Methods based on inter-scale dependencies:
  - EZW (Embedded Zerotrees of Wavelet coefficients),
  - SPIHT (Set Partitioning in Hierarchical Trees)
  - Tree-based representation of dependencies
  - Advantages: good exploitation of inter-scale dependencies, low complexity
  - Disadvantage: no resolution scalability
- Methods not based on inter-scale dependencies
  - Explicit bit-rate allocation among subbands
  - Entropy coding of coefficients
  - Advantages: Good exploitation of intra-scale dependencies, random access, resolution scalability
  - Disadvantage: no exploitation of inter-scale dependencies

Embedded Zerotrees of Wavelet coefficients

Main characteristics

- Quality scalability (i.e. progressive representation)
- Lossy-to-lossless coding
- Small complexity
- Rate-distortion performance much better than JPEG **above all at small rates**

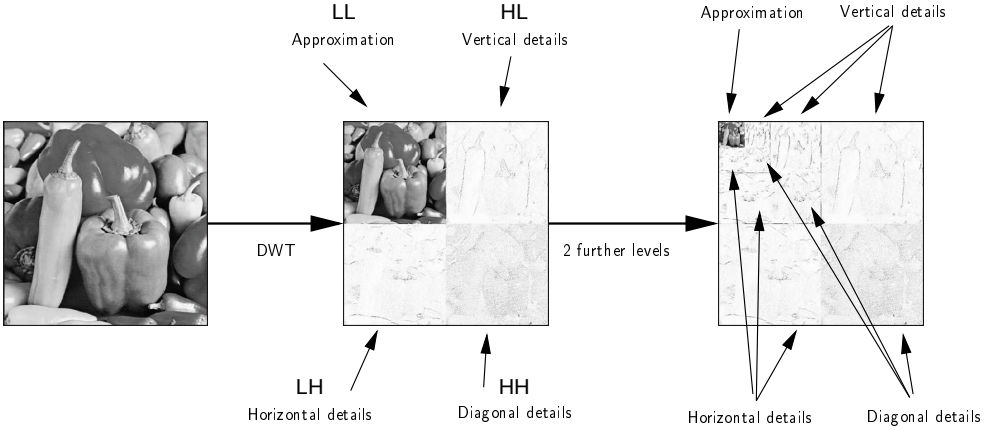
Progressive representation of DWT coefficients

Each new coding bit:
must convey the maximum of information

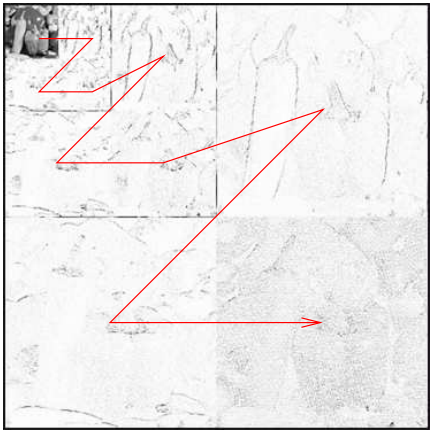
must reduce distortion as much as possible

- Send the largest coefficients first!
- Problem: localization overhead

Example: an image and its 2D-DWT



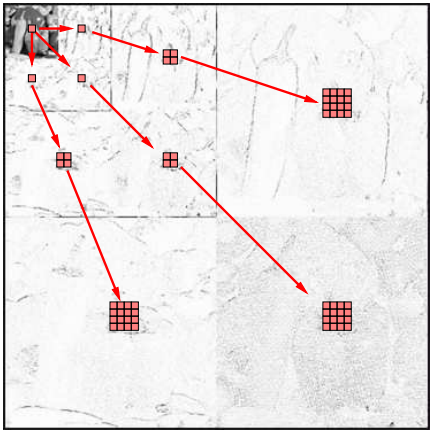
Progressive representation: SB order



EZW Algorithm

- The subband scan order alone is not enough to assure that largest coefficients are sent first
- We need to localize the largest coefficients
- Without having to send explicit localization information
- Idea: to exploit the inter-band correlation to predict the position of non-significant coefficients
- If the prediction is correct we save many coding bits (for all the predicted coefficients)

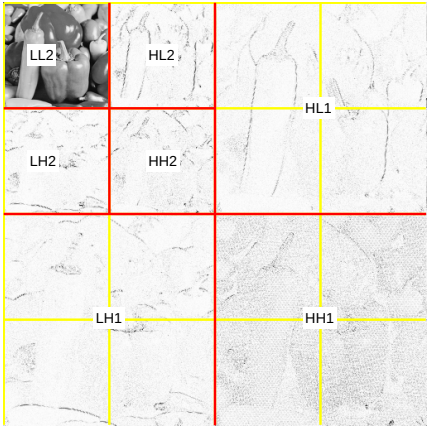
Zero-tree of wavelet coefficients

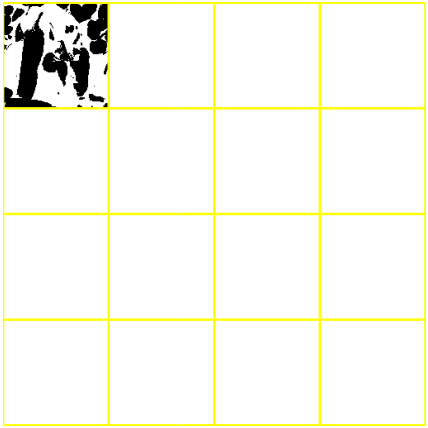


We consider **trees** of transform coefficients.
The root is a coefficient in the LL subband: it has three descendants, one in each of the first resolution detail subband.
From the second level on, each coefficients has four descendants: the colocated coefficients in the same orientation subband of the next resolution level.
We expect that a "small" coefficient (i.e., less than a threshold) in a SB has small descendents, since it does not correspond to an anomaly (object border).
A (sub-)tree of below-the-threshold coefficients is called a **zero-tree**
We expect zero-trees to be relatively frequent, thus we use a single symbol to encode a full zero-tree (possibly made up of many coefficients)

Transform coefficients bitplanes

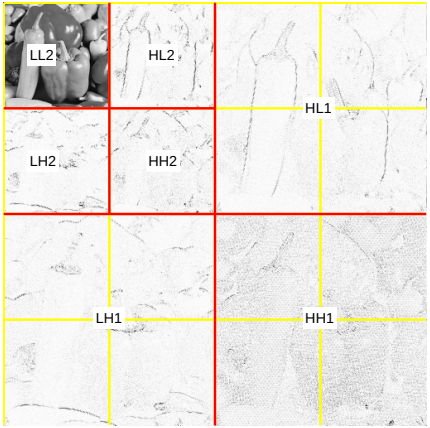
Most significant bitplane

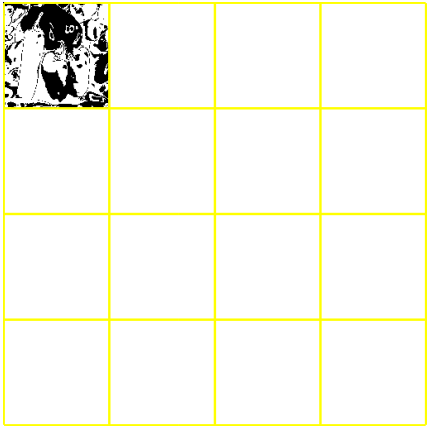




Transform coefficients bitplanes

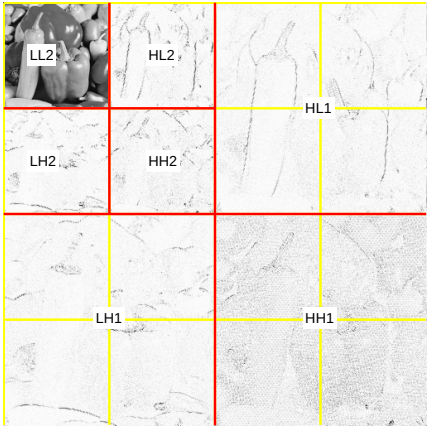
Second bitplane

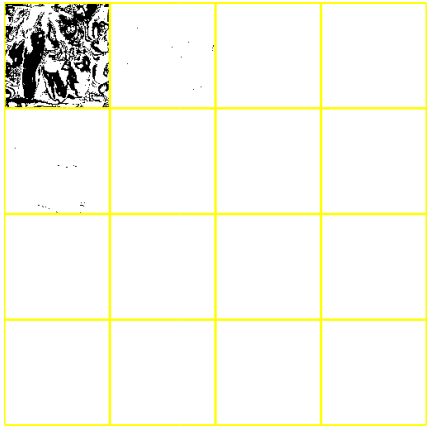




Transform coefficients bitplanes

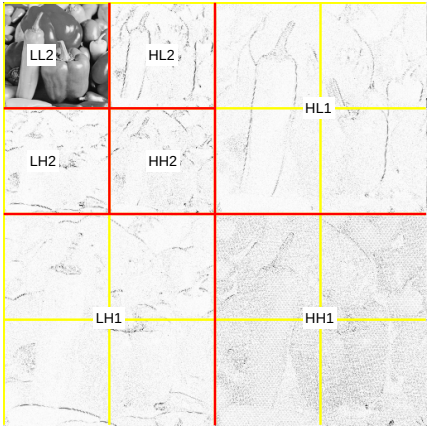
Third bitplane

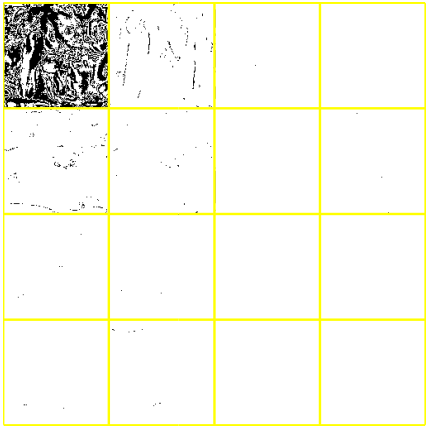




Transform coefficients bitplanes

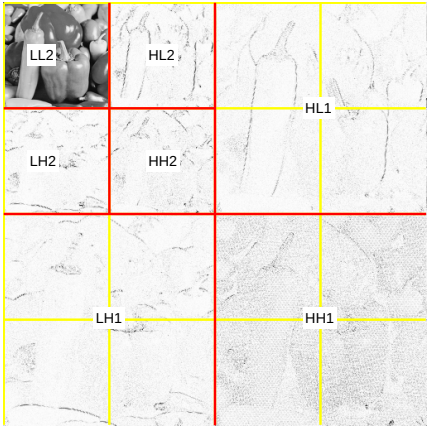
Fourth bitplane

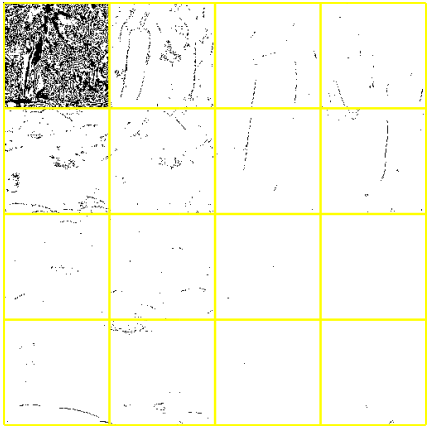




Transform coefficients bitplanes

Fifth bitplane





EZW idea

- **Auto-similarity** : When a coefficient is small (below a threshold) it is probable that its descendants are small as well
- In this case we use a single coding symbol to represent the coefficient and all its descendants. If c and all its descendants are smaller than the threshold, c is called a zero-tree root
- With just one symbol, (ZR) we code $(1 + 4 + 4^2 + \ldots + 4^{N-n})$ coefficients
- The localization information is implicit in the significance information

EZW algorithm

1. $k = 0$
2. $n = \lfloor \log_2 (|c|_{\max}) \rfloor$
3. $T_k = 2^n$
4. Let $\mathcal{L}$ be the list of all the DWT coefficients, according to the SB scan order (each SB in raster scan)
5. Let $\mathcal{S} = \emptyset$ be the list of significant coefficients
6. while (rate $<$ available rate)
   - Dominant pass
   - Refining pass
   - $T_{k+1} \leftarrow T_k/2$
   - $k \leftarrow k + 1$
7. end while

Dominant pass

1. Until $\mathcal{L}$ is not empty
   - Let c be the first coefficient of the list
   - **If** $c > T_k$, encode c as SP (Significant Positive) and put it in $\mathcal{S}$
   - **Else if** $c < -T_k$, encode c as SN (Significant Negative) and put it in $\mathcal{S}$
   - **Else if** no descendant is in abs value larger than T
     - Encode c as zero-tree root (ZR)
     - Remove **all its descendants from** $\mathcal{L}$
   - **Else** encode c as Isolated Zero (IZ)
2. Remove c from $\mathcal{L}$

Refining pass

1. Let $b = \log_2(T_k)$ the current bit plane index
2. For all c in $\mathcal{S}$, encode the b-th bit of its binary representation

Iteration and termination

- The k-th dominant pass allows to encode the b-th bit-plane
- A significant coefficient c is such that $2^k \leq |c| < 2^{k+1}$
- For the next step we halve the threshold: it is equivalent to pass to the next bitplane
- Algorithm stops when
  - the bit budget is exhausted; or when
  - all the bitplanes have been coded

EZW Algorithm: summary

- Bitplane coding: at the k-th pass, we encode the bitplane $\log_2 T_k$
- Progressive coding: each new bitplane allows refining the coefficients quantization
- Lossless coding of significance symbols
- Lossless-to-lossy coding: When an integer transform is used (i.e., al the transform operation can be performed without precision errors), and all the bitplanes are coded, the original image can be restored with zero distortion

EZW Encoding: Bitplane 1 ($T = 16$)

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Dominant Pass

$|26| \geq 16 \rightarrow$ **Significant Positive (SP)**

Symbol stream

EZW Encoding: Bitplane 1 ($T = 16$)

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Dominant Pass

$|26| \geq 16 \rightarrow$ **Significant Positive (SP)**

Symbol stream (SP, SN, ZR, IZ are arithmetically encoded):

SP										

EZW Encoding: Bitplane 1 ($T = 16$)

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Dominant Pass

$|6| < 16$ and all descendants $\{-13, 10, 6, 4\}$ have abs value $< 16 \rightarrow$ **Zero-Tree Root (ZR)**

Symbol stream (SP, SN, ZR, IZ are arithmetically encoded):

SP									

EZW Encoding: Bitplane 1 ($T = 16$)

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Dominant Pass

$|6| < 16$ and all descendants $\{-13, 10, 6, 4\}$ have abs value $< 16 \rightarrow$ **Zero-Tree Root (ZR)**

Symbol stream (SP, SN, ZR, IZ are arithmetically encoded):

SP	ZR								

EZW Encoding: Bitplane 1 ($T = 16$)

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Dominant Pass

-7 and 3 are also ZR. Now we have encoded the significance for all the coeff's

Symbol stream (SP, SN, ZR, IZ are arithmetically encoded):

SP	ZR	ZR	ZR							

EZW Encoding: Bitplane 1 ($T = 16$)

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Refinement Pass

Write the bit in position $\log_2 T = 4$ for all the significant coefficients: $26_{10} = 11010_2 \rightarrow 1$

Symbol stream (SP, SN, ZR, IZ are arithmetically encoded):

SP	ZR	ZR	ZR	1						

EZW Encoding: Bitplane 2 ($T = 8$)

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Dominant Pass

$|6| < 8$, but descendant $13 \geq 8 \rightarrow$ **Isolated Zero (IZ)**

Symbol stream (SP, SN, ZR, IZ are arithmetically encoded):

SP	ZR	ZR	ZR	1							

EZW Encoding: Bitplane 2 ($T = 8$)

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Dominant Pass

$|6| < 8$, but descendant $13 \geq 8 \rightarrow$ **Isolated Zero (IZ)**

Symbol stream (SP, SN, ZR, IZ are arithmetically encoded):

SP	ZR	ZR	ZR	1	IZ							

EZW Encoding: Bitplane 2 ($T = 8$)

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Dominant Pass

Next: -7, 3 are also scanned. Since $|c| < 8$ and no desc. ≥ 8, they stay ZR.

Symbol stream (SP, SN, ZR, IZ are arithmetically encoded):

SP	ZR	ZR	ZR	1	IZ	ZR	ZR					

EZW Encoding: Bitplane 2 ($T = 8$)

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Dominant Pass

Subband Level 2 Scan: $|13| \geq 8 \rightarrow$ **Significant Negative (SN)**

Symbol stream (SP, SN, ZR, IZ are arithmetically encoded):

SP	ZR	ZR	ZR	1	IZ	ZR	ZR	SN				

EZW Encoding: Bitplane 2 ($T = 8$)

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Dominant Pass

Subband Level 2 Scan: $|10| \geq 8 \rightarrow$ **Significant Positive (SP)**

Symbol stream (SP, SN, ZR, IZ are arithmetically encoded):

SP	ZR	ZR	ZR	1	IZ	ZR	ZR	SN	SP			

EZW Encoding: Bitplane 2 ($T = 8$)

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Dominant Pass

Subband Level 2 Scan: Next coeff's $< 8 \rightarrow$ **ZR**

Symbol stream (SP, SN, ZR, IZ are arithmetically encoded):

SP	ZR	ZR	ZR	1	IZ	ZR	ZR	SN	SP	ZR	ZR	

EZW Encoding: Bitplane 2 ($T = 8$)

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Refinement Pass

Write the bit in position $\log_2 T = 3$ for all the significant coeff's abs. values:

$26_{10} = 11010_2 \rightarrow 0 \quad 13_{10} = 1101_2 \rightarrow 1 \quad 10_{10} = 1010_2 \rightarrow 0$

Symbol stream (SP, SN, ZR, IZ are arithmetically encoded):

SP	ZR	ZR	ZR	1	IZ	ZR	ZR	SN	SP	ZR	ZR	0	1	0

EZW Encoding: Bitplane 3 ($T = 4$)

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Dominant Pass

$6 \geq 4, \rightarrow$ **SP**, $-7 \leq -4, \rightarrow$ **SN**

Symbol stream (SP, SN, ZR, IZ are arithmetically encoded):

SP	ZR	ZR	ZR	1	IZ	ZR	ZR	SN	SP	ZR	ZR	0	1	0	SP	SN

EZW Encoding: Bitplane 3 ($T = 4$)

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Dominant Pass

$3 < 4$ and no desc. ≥ 4, it stays ZR.

Symbol stream (SP, SN, ZR, IZ are arithmetically encoded):

SP	ZR	ZR	ZR	1	IZ	ZR	ZR	SN	SP	ZR	ZR	0	1	0	SP	SN
ZR																

EZW Encoding: Bitplane 3 ($T = 4$)

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Dominant Pass

Subband Level 2 Scan: $6 \geq 4$, $\rightarrow$ **SP**, $4 \geq 4$, $\rightarrow$ **SP**

Symbol stream (SP, SN, ZR, IZ are arithmetically encoded):

SP	ZR	ZR	ZR	1	IZ	ZR	ZR	SN	SP	ZR	ZR	0	1	0	SP	SN
ZR	SP	SP														

EZW Encoding: Bitplane 3 ($T = 4$)

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Dominant Pass

Subband Level 2 Scan: $4 \geq 4$, $\rightarrow$ **SP**

Symbol stream (SP, SN, ZR, IZ are arithmetically encoded):

SP	ZR	ZR	ZR	1	IZ	ZR	ZR	SN	SP	ZR	ZR	0	1	0	SP	SN
ZR	SP	SP	SP													

EZW Encoding: Bitplane 3 ($T = 4$)

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Dominant Pass

Subband Level 2 Scan: The other 3 coeff's are non-significant

Symbol stream (SP, SN, ZR, IZ are arithmetically encoded):

SP	ZR	ZR	ZR	1	IZ	ZR	ZR	SN	SP	ZR	ZR	0	1	0	SP	SN
ZR	SP	SP	SP	ZR	ZR	ZR										

EZW Encoding: Bitplane 3 ($T = 4$)

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Refinement Pass

Write the bit in position $\log_2 T = 2$ for all the significant coeff's abs. values:
$26_{10} = 11010_2 \rightarrow 1$ $13_{10} = 1101_2 \rightarrow 0$ $10_{10} = 1010_2 \rightarrow 1$ $6_{10} = 110_2 \rightarrow 1$ $7_{10} = 111_2 \rightarrow 1$
$6_{10} = 110_2 \rightarrow 1$ $4_{10} = 100_2 \rightarrow 0$ $4_{10} = 100_2 \rightarrow 0$

Symbol stream (SP, SN, ZR, IZ are arithmetically encoded):

SP	ZR	ZR	ZR	1	IZ	ZR	ZR	SN	SP	ZR	ZR	0	1	0	SP	SN
ZR	SP	SP	SP	ZR	ZR	ZR	1	0	1	1	1	1	0	0	...	

EZW Decoding

Decoded:

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

True:

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Decoding

Initial state: All coefficients are unknown (00000_2).

Bitstream Progress:

EZW Decoding

Decoded:

24	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

True:

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Dominant Pass 1 ($T = 16$)

Received **SP** at [1,1].
The decoder knows the MSB: $1xxxx_2$.
Choosing the center of $\{10000_2 \dots 11111_2\} \rightarrow 11000_2$ (**24**).

Bitstream Progress: D1: [SP, ZR, ZR, ZR]

EZW Decoding

Decoded:

28	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

True:

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Refinement Pass 1 ($T = 16$)

Received bit **1**.

The value is now $11xxx_2$.

Choosing the center of $\{11000_2 \dots 11111_2\} \rightarrow 11100_2$ (**28**).

Bitstream Progress: D1: [SP, ZR, ZR, ZR] R1: [1]

EZW Decoding

Decoded:

28	0	-12	0
0	0	0	0
0	0	0	0
0	0	0	0

True:

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Dominant Pass 2 ($T = 8$)

Received **SN** at [1,3].
The decoder knows it is $-01xxx_2$.
Choosing the center of $\{01000_2 \dots 01111_2\} \rightarrow 01100_2$ (**-12**).

Bitstream Progress: D1: [SP...], R1: [1], D2: [IZ, ZR, ZR, SN, SP, ZR, ZR]

EZW Decoding

Decoded:

28	0	-12	12
0	0	0	0
0	0	0	0
0	0	0	0

True:

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Dominant Pass 2 ($T = 8$)

Received **SP** at [1,4].
The value is $01xxx_2$.
Reconstruction: 01100_2 (**12**).

Bitstream Progress: D1: [SP...], R1: [1], D2: [IZ, ZR, ZR, SN, SP, ZR, ZR]

EZW Decoding

Decoded:

26	0	-14	10
0	0	0	0
0	0	0	0
0	0	0	0

True:

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Refinement Pass 2 ($T = 8$)

Received bits **0, 1, 0**.

- [1,1]: $110xx_2 \rightarrow \text{mid}\{11000, 11011\} = 11010_2$ (**26**)
- [1,3]: $011xx_2 \rightarrow \text{mid}\{01100, 01111\} = 01110_2$ (**-14**)
- [1,4]: $010xx_2 \rightarrow \text{mid}\{01000, 01011\} = 01010_2$ (**10**)

Bitstream Progress: D1, R1, D2, R2: [0, 1, 0]

EZW Decoding

Decoded:

26	6	-14	10
-6	0	0	0
0	0	0	0
0	0	0	0

True:

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Dominant Pass 3 ($T = 4$)

Received **SP**, **SN**, **ZR** for the SBs at level 1.
The value for significant coeff's is $\pm 001xx_2$.
Reconstruction: $\pm 00110_2$ ($\pm$**6**).

Bitstream Progress: D1, R1, D2, R2, D3: [SP, SN, ZR, SP, SP, SP, ZR, ZR, ZR]

EZW Decoding

Decoded:

26	6	-14	10
-6	0	6	6
6	0	0	0
0	0	0	0

True:

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Dominant Pass 3 ($T = 4$)

Received **SP**, **SP**, **SP** for the next 3 coeff's at level 2.
Reconstructed as 00110_2 (**6**).

Bitstream Progress: D1, R1, D2, R2, D3: [SP, SN, ZR, SP, SP, SP, ZR, ZR, ZR]

EZW Decoding

Decoded:

27	7	-13	11
-7	0	7	5
5	0	0	0
0	0	0	0

True:

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Refinement Pass 3 ($T = 4$)

Refinement bits for significant coefficients

Bitstream Progress: D1, R1, D2, R2, D3: [SP, SN, ZR, SP, SP, SP, ZR, ZR, ZR],R3

JPEG2000

- JPEG2000 aims at challenges unresolved by previous standards:
- Low bit-rate coding: JPEG has low quality for $R < 0.25$ bpp
- Synthetic images compression
- Random access to image parts
- Quality and resolution scalability

New functionalities

- Region-of-interest (ROI) coding
- Quality and resolution scalability
- Tiling
- Exact coding rate
- Lossy-to-lossless coding

Algorithm

JPEG2000 is made up of two *tiers*

- First tier
  - DWT and quantization
  - Lossless coding of codeblocks
- Second tier
  - **EBCOT**: embedded block coding with optimized truncation
  - Scalability (quality, resolution) and ROI management

Quantization in JPEG2000

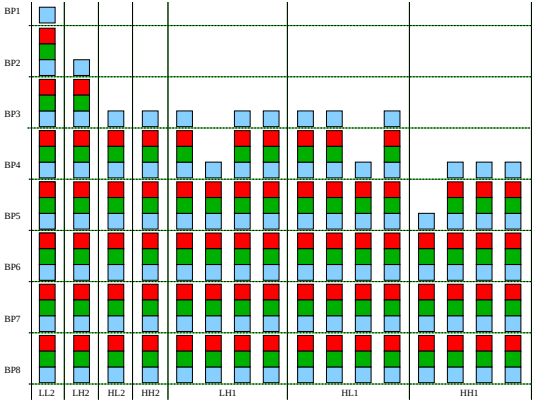
- DWT coefficients are encoded with a very fine quantization step
- For the lossless coding case, DWT coefficients are integers, and they are not quantified
- In summary, it is not in the quantization step that the really lossy operations are performed
- The lossy coding is performed by the bitstream truncation of Tier 2

EBCOT

Embedded Block Coding with Optimized Truncation

- Each subband is split in equally sized blocks of coefficients, called codeblocks
- The codeblocks are losslessly and independently coded with an arithmetic coder
- We generate as many bitstreams as codeblocks in the image

Example: bitstreams & codeblocks



For each codeblock, we produce a bitstream (the columns in this graph). The bitstream of each codeblock can be partitioned according to the bitplane. JPEG2000 lossless coding is structured in such a way to encode a bitplane in three passes, which allows for a fine potential bitstream partitioning.

EBCOT

Optimization

- If we keep all the bitstreams of all the codeblocks, we end up with a huge bitrate
- We have to truncate the bitstream to attain the target bit-rate
- Problem: how to truncate the bitstreams with a minimum resulting distortion?

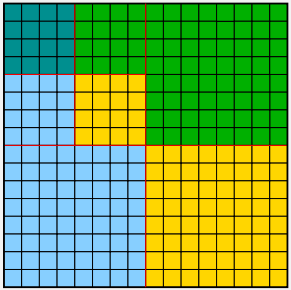
$$\min \sum_i D_i \qquad \text{subject to} \qquad \sum_i R_i \leq R_{\text{tot}}$$

- Solution : Lagrange multiplier

$$J = \sum_i D_i + \lambda \left(\sum_i R_i - R \right)$$

EBCOT

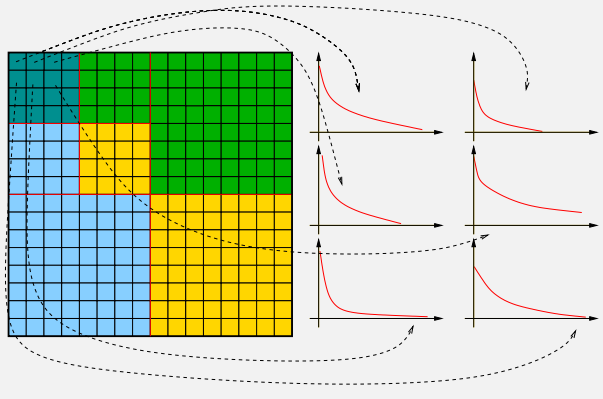
Rate-distortion curve per each codeblock



After Tier 1, we know the RD curves of all the codeblocks: $D_i = D_i(R_i)$

EBCOT

Rate-distortion curve per each codeblock



After Tier 1, we know the RD curves of all the codeblocks: $D_i = D_i(R_i)$

EBCOT

Embedded block coding with optimized truncation

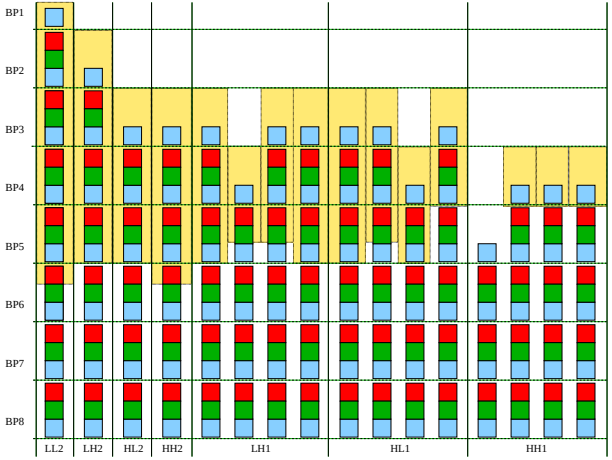
- Optimal truncation point:

$$\frac{\partial J_i}{\partial R_i} = 0 \qquad \frac{\partial J_i}{\partial R_i} = \frac{\partial D_i}{\partial R_i} + \lambda = 0$$
$$\frac{\partial D_i}{\partial R_i} = -\lambda$$

- The optimal truncation point (R_i^*) for block i is such that the slope of $D_i(R_i)$ is the same as other codeblocks
- The value of the Lagrange multiplier can be find by an iterative algorithm.
- We can have several truncation points for as many target rates (quality scalability)

Example of bit-rate allocation

Allocation for a given bit-rate, maximal quality and resolution

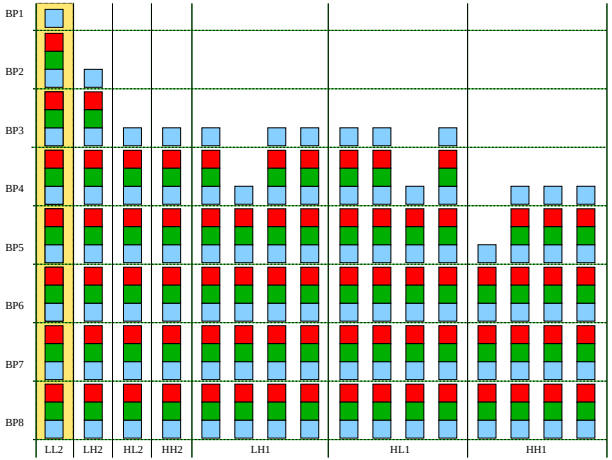


For example, for a given target rate, EBCOT will tell which bitplane must be encoded for each codeblock. This strategy insures at the same time:

1. To reach the exact target bitrate (with an error usually less than 0.1%
2. To achieve the minimal MSE for that bitrate

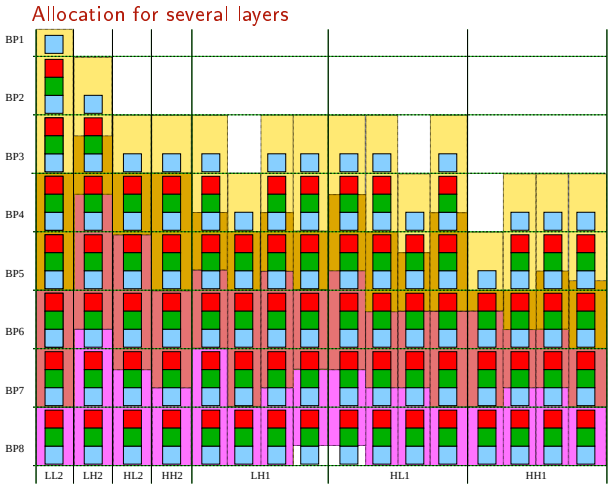
Example of bit-rate allocation

Allocation for maximal quality and minimal resolution



We can also decide for different rate allocation, as, for example, minumum resolution (only LL sub-band), but max quality (all bitplanes)

Example of bit-rate allocation



We can compute the optimal truncation points for several target bit-rates, and organize the encoded bitstream in quality layers: each layer corresponds to a color in this example. Special syntax elements (markers) are used to recognize the layer starting and final points in the bitstream.

Comparison JPEG / JPEG2000

Image Originale, 24 bpp

Comparison JPEG / JPEG2000

Rate: 1bpp

Comparison JPEG / JPEG2000

Rate: 0.75bpp

Comparison JPEG / JPEG2000

Rate: 0.5bpp

Comparison JPEG / JPEG2000

Rate: 0.3bpp

Comparison JPEG / JPEG2000

Rate: 0.2bpp

Comparison JPEG / JPEG2000

Rate: 0.2bpp for JPEG, 0.1bpp for JPEG2000

Error robustness in compressed data

- Transmission and storage operations introduce errors in files with a certain probability
- A wrong bit in an uncompressed image only affects the color of one pixel
- Compressed streams are much more vulnerable:
  - Predictive and differential coding introduce data dependency, implying error propagation
  - Variable lenght coding implies possibly decoding drift
  - Errors on headers and metadata are particularly dangerous

Trade-off between robustness and rate

Error robustness can be achieved by several strategies:

- Error correction codes are very effective but demand a considerable increase of the coding rate: they are typically used only on small, sensitive data such as heades and metadata
- Error robustness of compressed signals is achieved by using markers that allow to recover the synchronization of the lossless code and stop error propagation
- Markers are periodically inserted into the bit-stream: increasing the period improves robustness because stops earlier the propoagation of errors.
- However, using markers increase the file size and also request marker emulation preventions strategies, usually achieved with bit stuffing, with further file size increase

Error robustness: JPEG2000 vs JPEG

JPEG2000 is inherently more robust than JPEG

- Data priorization is possible to protect headers
- No dependency among codeblocks
  - Much less error propagation
- No block-based transform
  - No *blocking* artifacts

Error effect: JPEG

Left: JPEG, $p_E = 10^{-4}$
Right: JPEG, $p_E = 10^{-4}$

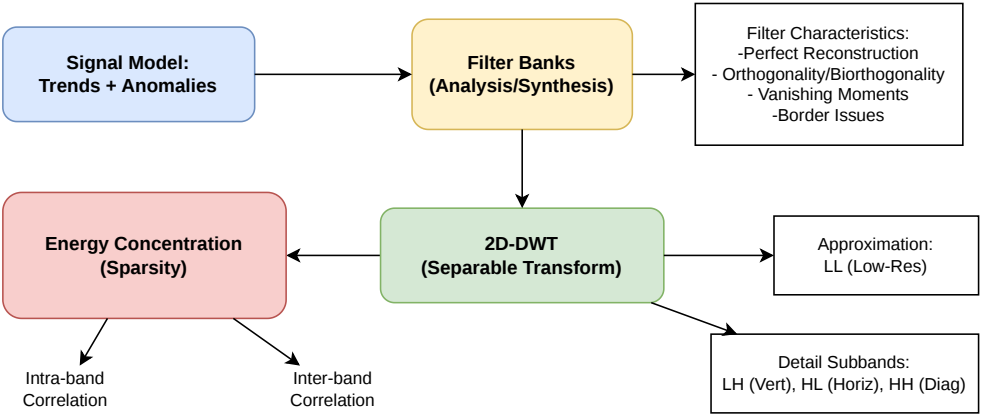
Error effect: JPEG and JPEG 2000

Left: JPEG, $p_E = 10^{-4}$
Right: JPEG 2000, $p_E = 10^{-4}$

Error effect: JPEG and JPEG 2000

Left: JPEG, $p_E = 10^{-3}$
Right: JPEG 2000, $p_E = 10^{-3}$

Conclusions: DWT conceptual map



Conclusions: EZW/JP2K conc. map